

CSC 648/848 Software Engineering Spring 2026

Pathwise

Section 2

Team 2

Omeid Nadery — Team Lead — onadery@sfsu.edu

Jason Le — Back End Lead — jle34@sfsu.edu

Damian Perez — Front End Lead — dperez52@sfsu.edu

Brandon Sanchez — Bsanchez15@sfsu.edu

Laura Wong — lwong25@sfsu.edu

Mikias Berhane — mberhane1@sfsu.edu

Cielina Maree Lubrino — clubrino@sfsu.edu

Milestone 4

4/21/2026

History Table (Revision)

Name	Revision Summary	Revision Date
Laura	Started on <i>Executive Summary</i>	2/10/2026
Omeid	Completed <i>Initial List of Functional Requirements</i> and copied <i>Non-Functional Requirements</i>	2/10/2026
Damian	Started on <i>Main Data Items and Entities</i>	2/12/2026
Cielina	Started on <i>Personas and Use Cases</i>	2/12/2026
Brandon	Started <i>High-level System Architecture and Technologies Used</i>	2/13/2026
Jason Le	Started <i>Competitive analysis</i>	2/14/2026
Damian	Finished <i>Main Data Items and Entities</i> ; updated document formatting	2/17/2026
Laura	Finished <i>Executive Summary</i>	2/17/2026
Brandon	Finished <i>High-level System Architecture and Technologies Used</i>	2/17/2026
Mikias	Revised <i>Competitive Analysis</i>	2/17/2026
Omeid	Modified functional requirements and grouped them in priorities	3/1/2026
Cielina	Added <i>UML Diagrams</i>	3/2/2026
Damian	Added completed wireframes and storyboard to <i>UI Mockups and Storyboard</i>	3/3/2026
Cielina	Added completed wireframes and storyboard to <i>UI</i>	3/3/2026

	<i>Mockups and Storyboard</i>	
Brandon	Added completed wireframes and storyboard to <i>UI Mockups and Storyboard</i>	3/3/2026
Damian	Added further business value justification to <i>Executive Summary</i>	3/6/2026
Mikias	Finished the <i>Risks & Project Management</i>	3/6/2026
Omeid	Added Appendix I and finished High Level Architecture, DB Organization	3/7/2026
Damian	Added Frontend Team Support to <i>Teams and Roles</i>	3/9/2026
Omeid	QA test plan done AND Self Check Non-Func Reqs.	4/16/2026
Damian	Added Test Objectives and Test Background/Setup to <i>Usability Test Plan</i> for search feature	4/17/2026
Cielina	Added Product Summary	4/17/2026
Laura	Added Usability Task Description to <i>Usability Test Plan</i> for search feature	4/19/2026
Jason	Added description for the overall coding style	4/19/2026
Damian	Edited <i>Usability Test Plan</i> section	4/20/2026
Brandon	Added self check security	4/20/2026
Jason	Added personal review of the code snippet from QA	4/20/2026

1) Product Summary

Pathwise is a web-based academic planning platform designed to help college students organize their goals, track progress, and discover resources in one place. Students can create structured goals broken down into projects and milestones, monitor their progress through a personalized dashboard, and search for curated resources and templates by category, content type, skill area, and AI-enabled status.

Registered users can bookmark resources for later, while guest users can still browse publicly available content without an account. The application URL can be accessible at

<http://3.135.184.72/vertical-prototype/index.html>

Committed Priority 1 Functions:

1. Users can register, log in, and log out with secure session-based authentication.
2. Registered users can create, edit, delete, and categorize goals with a title, description, category, target date, and status.
3. Users can change a goal's status (active, paused, or completed) with a single button click.
4. Users can create projects linked to a specific goal.
5. Each project contains milestones with a title, description, target date, and completion status.
6. Users can attach resources, templates, or workflows to a specific goal or project.
7. A keyword-based search engine lets users find resources, templates, and workflows.
8. Users can filter search results by category, content type, skill area, and AI-enabled status.
9. Search results display relevant metadata including tags, ratings, and descriptions.
10. Registered users can bookmark and save resources for later access.
11. A personalized dashboard shows a user's active goals, projects, upcoming milestones, and saved resources after login.
12. Goals and milestones display visual progress indicators such as progress bars or status labels.
13. Guest users can browse publicly available categorized content without registering.

2) Usability Test Plan

Test Objectives

This usability test evaluates the Pathwise search and resource discovery feature. The goal is to determine whether users can easily search for resources, understand the results, and feel confident using the feature. Search was selected because it is one of the main user-facing functions of Pathwise and plays an important role in helping users find academic, career, wellness, and personal development resources. For Milestone 4, the main focus of this test is user satisfaction, which will be measured with a short Likert-scale questionnaire after the task is completed.

Test Background and Setup

The usability test will be conducted on the deployed Pathwise application using the search feature. The tester will begin on the main search page and use the search bar and available filters to locate resources based on a keyword or topic. The system setup for this test is a desktop or laptop computer with internet access and a modern web browser such as Google Chrome or Microsoft Edge. The application URL to be tested is: <http://3.135.184.72/vertical-prototype/index.html>

The intended users are students or similar users looking for resources related to academics, career planning, wellness, or personal growth. The test assumes the user has basic web browsing experience but no special training on Pathwise. The main usability factor being measured is user satisfaction, including whether the search process feels clear, easy to use, and helpful.

Usability Task Description

Begin on the Pathwise search page. Enter a keyword related to a resource need, such as study support or wellness, and review the results returned by the system. If needed, use the available filters to narrow the results. After completing the task, answer the Likert-scale questions based on how clear, easy, and useful the search experience felt.

Task	Description
Task	Search with term “study”
Machine State	Start on the Pathwise browse page
Successful completion criteria	Study is typed in search bar and searched
Benchmark	Completed in 10 seconds or under

Task	Description
Task	Search wellness type categories
Machine State	Start on the Pathwise browse page
Successful completion criteria	Opened the drop down menu and selected wellness in category before searching

Benchmark	Completed in 10 seconds or under
-----------	----------------------------------

Task	Description
Task	Search for template type resources
Machine State	Start on the Pathwise browse page
Successful completion criteria	Opened the drop down menu and selected template in type category before searching
Benchmark	Completed in 10 seconds or under

Task	Description
Task	Filter to only internship
Machine State	Start on the Pathwise browse page with all results
Successful completion criteria	Checked box next to internship in tags
Benchmark	Completed in 10 seconds or under

Task	Description
Task	Filter free resources
Machine State	Start on the Pathwise browse page with all results
Successful completion criteria	Opened cost drop down and selected free option
Benchmark	Completed in 10 seconds or under

Task	Description
Task	Search for AI-enabled resources
Machine State	Start on the Pathwise browse page with all results
Successful completion criteria	Checked box next to AI-enabled resources only option
Benchmark	Completed in 10 seconds or under

How Effectiveness Will Be Measured

The effectiveness will be measured by if the testers successfully complete the task within the benchmark time of 10 seconds. We will see what percentage completes each of the tasks and the amount that fails. The data will be recorded in a table detailing the test/use case, % completed, errors per task and comments.

How Efficiency Will Be Measured

Efficiency will be measured in time and number of clicks. This can be done within the app to be measured automatically. The less time and clicks taken, the better.

Likert Subjective Test

The options provided for filters and drop down menus were helpful. (check one)

☐ strongly disagree ☐ disagree ☐ neutral ☐ agree ☐ strongly agree

Comments:

It was easy to use the search features. (check one)

☐ strongly disagree ☐ disagree ☐ neutral ☐ agree ☐ strongly agree

Comments:

I like how the search page looks. (check one)

☐ strongly disagree ☐ disagree ☐ neutral ☐ agree ☐ strongly agree

Comments:

3) QA test plan

Test Objectives)

This QA test plan verifies that the Pathwise Search/Resource Discovery function works according to system requirements (functional QA, not usability). This testing checks whether keyword search, filtering, and guest access behavior produce correct and consistent outputs across two different browsers.

HW and SW Setup)

- Hardware: Personal laptop/desktop (8GB+ RAM, stable internet)
- Operating System: Windows 10/11
- Browsers Tested:
 - Google Chrome
 - Microsoft Edge
- Application URL: **http://3.135.184.72/vertical-prototype/index.html**
- User Contexts:
 - Signed in or Guest User

Feature to be Tested)

- Feature: Pathwise Search / Resource Discovery
- Scope tested: keyword search over discovery content, filter behavior (category + AI-enabled), default behavior for empty query, and guest browsing restrictions related to save/bookmark actions.

Test Table Below:

QA Test Table)

Test #	1	2	3
Test Title	Positive keyword search	Filter validation (Category + AI-enabled)	Edge case: empty query + no filters
Test Description	Validate keyword search returns relevant public discovery items.	Validate that combined filters restrict the result set correctly.	Validate default discovery behavior on empty submission.
Test Input	In the search bar, enter “resume” with no filters, then click Search.	Blank search, set Category = Career, AI-Enabled = Yes, click Apply/Search.	Clear search box and all filters; submit search.
Expected Correct Output	(1) Results list is displayed. (2) Each result contains resume (case-insensitive) in title, description, or tag. (3) Results shown are public discovery items only.	(1) All returned cards are in Career category. (2) Every returned card shows AI-enabled indicator. (3) No non-matching category card appears.	(1) System returns default public discovery list (not validation error). (2) Page remains functional and responsive. (3) If no data exists, empty-state message appears instead of failure.
Google Chrome Results	Pass	Pass	Pass
Microsoft Edge Results	Pass	Pass	Pass

4) Code Review:

Coding Style:

No formal adopting of any specific structured format. In terms of syntax, standard two space indents, semicolons, and camel case for the names of variables and functions. There are minimal comments overall and are only there for more complex code blocks and or instructions of setting up personal environments. The overall style is more in line of writing projects for small teams and codebases without much thought into further scalability beyond project scope. Could use a more structured format for easy readability and future scalability for new team members.

Code block for peer review:

[application/server.js](#)

Our QA targeted the search/find resource in the [server.js](#) file in our repository from lines 133 to 357. The code itself runs really well as it has good input validation, and has good placeholders for querying. It distinguishes logged on users and guest accounts. There are also empty result handlers and refreshed schemas that prevent catastrophic failure when searching. There are very few comment blocks that would quickly clarify what some parameters mean and how they are used. There are only one line comments for overall description even for more complicated sections. This leads to poor documentation for both new and old team members to catch up on. This could be mitigated with both a file header and function header to explain the elements of the file. While there are useful comments, it still needs major improvement. It doesn't meet the general standard of any industry, computer science veteran, or any professor at SFSU. The code itself is functional and serves the purpose well though. Overall just needs improved commenting.

5) Self-check on best practices for security

Major Assets Protected:

1. User passwords
 - a. Encrypted in the database using bcrypt. Passwords are hashed and never transmitted in plaintext.
2. User account data
 - a. Protected by session-based authentication with secure session cookies. Users must pass middleware to access personal data.
3. User bookmarks and saved resources
 - a. Database-level access control: Bookmarks are tied to user_id. Only the logged-in user can view/delete their own bookmarks
4. Session authentication tokens
 - a. Session-based with express-session using secure cookies. Frontend sends credentials: 'include' on all API calls. SESSION_SECRET stored in .env. The env is never committed to github
5. Submitted resource data
 - a. Users can only submit resources if authenticated. It is defaulted to private so only user can see unless faculty or staff.
6. File upload security
 - a. Images limited to 4MB max file size. Files stored with random filename to prevent path traversal attacks.
7. DataBase access
 - a. All queries use prepared statements with (?) as placeholder to prevent sql injections

Password Encryption in Database:

CONFIRMED — Bcrypt hashing with 10 salt rounds in routes/auth.js. Passwords stored as password_hash, never plaintext.

Code: `const passwordHash = await bcrypt.hash(pw, BCRYPT_ROUND);` where BCRYPT_ROUND = 10 in routes/auth.js

Input Data Validation:

1. Search bar input — Limited to 1-40 alphanumeric characters and spaces only
 - a. Code: Regex validation `/^[a-z0-9 ]{1,40}$/i` in vertical-prototype/app.js
2. Resource title — Must be 1-255 characters
 - a. Code: `if (titleStr.length < 1 || titleStr.length > 255)` in routes/resources.js
3. Required fields — Title, description, and category_id are mandatory
 - a. Code: `if (!title || !description || categoryIdRaw === undefined)` in routes/resources.js
4. Category ID — Must be a valid integer matching database categories
 - a. Code: `parseInt(categoryIdRaw, 10)` with NaN check in routes/resources.js
5. Cost field — Truncated to maximum 32 characters
 - a. Code: `.trim().slice(0, 32)` in routes/resources.js
6. All user inputs — Trimmed of whitespace before storage
 - a. Code: `String(input).trim()` applied to title, description, URL, and cost in routes/resources.js

6) Self-check: Adherence to original Non-functional specs – performed by team leads

1. Application shall be developed, tested, and deployed using tools and servers approved by Class CTO and as agreed in M0.	DONE
2. Application shall be optimized for standard desktop/laptop browsers and must render correctly on the two latest versions of at least two major browsers.	DONE
3. Selected application functions must render well on mobile devices (this is a plus).	ON TRACK
4. Data shall be stored in the team's chosen database technology on the team's deployment server.	DONE
5. Privacy of users shall be protected, and all privacy policies shall be clearly communicated to users.	DONE
6. The language used shall be English.	DONE
7. Application shall be very easy to use and intuitive.	DONE
8. Google Maps and Google Analytics shall be added where appropriate (e.g., campus resources, study locations, event discovery, usage analytics).	ON TRACK
9. No external e-mail clients shall be allowed. You shall use in-site webmail only.	DONE
10. Pay functionality, if any, shall not be implemented nor simulated in the UI.	DONE
11. Site security: basic best practices shall be applied (as covered in the class).	ON TRACK
12. Modern Software Engineering processes and practices shall be used as specified in the class, including collaborative and continuous software development.	DONE
13. The website shall prominently display the following exact text on all pages at the top of the WWW page: "SFSU Software Engineering Project CSC 648-848, Spring 2026. For Demonstration Only"	DONE
14. AI-powered assistant and intelligent search enhancements are optional (bonus requirements) and must be implemented modularly.	ON TRACK